

MTH 451/551

Lecture 3

Prof. W. Pazner

1 Floating Point Arithmetic

[Textbook reference: §1.4]

As discussed in the first lecture, the point of numerical analysis is **to use computer algorithms to solve mathematics problems**. Therefore, it is supremely important to understand how numbers are represented by the computer.

1.1 Prelude: Integer Representations

For integers, there is no real ambiguity. Integers are represented by the computer using their representation in binary. Typically, integers are stored using a fixed number of bits (for example, 32 or 64), which allows us to represent all integers in a finite range. A **bit** is simply a placeholder that can take the value 0 or 1. Representing integers beyond that range (i.e. with larger magnitude) simply requires using more bits. As an example, if we are encoding non-negative integers, we can represent the numbers 0, 1, 2, ..., 7 using three bits. This is a total of $8 = 2^3$ representable numbers, corresponding to ways to choose from $\{0, 1\}$ three times.

Number	Binary Representation
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

The number associated with the binary representation $b_2b_1b_0$ is $\sum_{k=0}^2 b_k 2^k$. If we wished to represent *signed* numbers (positive or negative), we would use one bit of information to encode the

sign (there are various ways of doing this). The upshot of this discussion is that it is relatively straightforward and uncontroversial how to represent integers on the computer.

1.2 Floating Point Numbers

As soon as we want to represent *real numbers* (or even just *rational numbers*), the situation becomes more complicated. As opposed to the integer case, where a given number of bits can represent all possible integers within a range, the same is fundamentally impossible for real numbers, because every interval contains infinitely many of them, and so they cannot all be representable with finite information. This means that, even when representing numbers within a given range, we need to consider issues of precision and rounding. **Floating point numbers** are the most commonly (almost universally) used method to represent these kinds of numbers on the computer.

Before we define floating point numbers, it is useful to contrast them with the conceptually simpler but generally less useful *fixed point numbers*. We can think of fixed point numbers as “fixing the decimal point” in an integer number, allowing us to represent fractional numbers. Essentially, this is the same as fixing a denominator d , and then just representing integers, which get multiplied by $1/d$. These types of numbers are often used in financial calculations, where quantities in dollars are represented using a denominator of $d = 100$, allowing the exact representation of all dollars-and-cents quantities within a range. However, outside of this and some other niche applications, fixed point numbers are relatively rarely used, in favor of the much more flexible *floating point numbers*. The key advantage of floating point numbers is that, essentially, the fractional multiplier is also encoded as part of the number. This way, rather than “fixing” the position of the decimal point, it is allowed to “float”.

Floating point numbers divide up the bits to represent three parts of the number: the sign, the mantissa, and the exponent. These terms correspond to representing a number as

$$x = (-1)^s \times m \times b^e,$$

where s encodes the **sign** (either 0 or 1), $m \in [1, b)$ is the **mantissa**, and e is the **exponent**; here, b is called the **base** of the representation. This is essentially the same as “scientific notation” for numbers, which you may be familiar with. So, to represent a floating point number, we need to encode s , m , and e . On the computer, it is natural to use **binary** (base two) representation, and we take $b = 2$.

The sign s is represented using a single bit. The number of bits used to represent the mantissa and exponent depends on the exact choice of representation. Luckily, these representations have been standardized in a standard called IEEE 754 that is basically universal among all computers. This standard defines how to represent floating point numbers with 32, 64, or 128 bits.

Total Bits	Sign	Mantissa	Exponent
32	1	23	8
64	1	52	11
128	1	112	15

32-bit floating point numbers are called **single precision**. Likewise, 64-bit floating point numbers are called **double precision**, and 128-bit floating point numbers are called **quadruple precision**. In numerical analysis, double precision is the most widely used. In other fields, such as machine learning, single precision (or even half precision, using 16 bits) is more widely used; similarly for computer graphics. In this course, we will focus on **64-bit double-precision floating point numbers**.

The exponent e is represented by 11 bits, so it can take on a total of $2^{11} = 2,048$ possible values; these give the range $-1022 \leq e \leq 1023$, along with two reserved values used for special numbers. The 52-bit mantissa $m \in [1, 2)$ is given by

$$m = 1 + \sum_{k=1}^{52} d_k 2^{-k} = 1.d_1 d_2 \cdots d_{52},$$

where d_k is the value of the k th mantissa bit. The sign bit s can be either 0 or 1. The number represented by these 64 bits is

$$y = (-1)^s \times m \times 2^e = \pm(1.d_1 d_2 \cdots d_{52}) \times 2^e.$$

We use the symbol $\mathbb{F}$ to denote the (finite) set of all such 64-bit numbers, together with the number zero. (*Note: these are the **normalized** floating-point numbers. Some special numbers, such as positive infinity and negative infinity, as well as so-called “subnormal” numbers, can also be represented in this format, but these are beyond the current scope.*)

The mantissa is always in the interval $[1, 2)$. We can be more precise. The smallest mantissa is when all its bits are zero, and is exactly equal to 1. The largest mantissa is when all its bits are one, and is equal to

$$1 + \sum_{k=1}^{52} 2^{-k} = \sum_{k=0}^{52} (2^{-1})^k = \frac{1 - 2^{-53}}{1 - 2^{-1}} = 2 - 2^{-52}.$$

The spacing between successive mantissa values, which is called **machine epsilon**, is

$$\epsilon = 2^{-52} = 2.220446049250313080847263336181640625 \times 10^{-16}.$$

The spacing between consecutive floating point numbers in $[2^e, 2^{e+1}]$ is $\epsilon \times 2^e$ (and likewise for $[-2^{e+1}, -2^e]$). For this reason, and as the next theorem will demonstrate, it is often said that double precision has (approximately) **16 (decimal) digits of precision**.

Another special value in $\mathbb{F}$ is called **unit roundoff**, and is given by

$$u = 2^{-53} = 1.1102230246251565404236316680908203125 \times 10^{-16}.$$

This number features in error analysis of rounding and floating point arithmetic. The smallest and largest positive numbers in $\mathbb{F}$ are

$$F_{\min} = 2^{-1022} \approx 2.2250738585072013830902327173324040642 \times 10^{-308},$$

$$F_{\max} = (1 - u)2^{1024} \approx 1.7976931348623157081452742373170435680 \times 10^{308}.$$

We say that a real number $x \in \mathbb{R}$ is **in the range of $\mathbb{F}$** if $F_{\min} \leq |x| \leq F_{\max}$ or if $x = 0$. For such an x , we define the **rounded value** of x to be the number $\text{fl}(x) \in \mathbb{F}$ nearest to x . In case of a tie, we round to the nearest element of $\mathbb{F}$ with $d_{52} = 0$ (we say that **ties round to even**; this is an unbiased rounding method).

The following theorem bounds the relative error between a number x in the range of $\mathbb{F}$ and its rounded value $\text{fl}(x)$, in terms of the unit roundoff u .

Theorem 1. *If $x \in \mathbb{R}$ is in the range of $\mathbb{F}$, then $\text{fl}(x) = (1 + \delta)x$ for some $|\delta| < u$. In other words, for non-zero x , the relative error in rounding is smaller than u ,*

$$\frac{|x - \text{fl}(x)|}{|x|} = |\delta| < u.$$